Ryerson University

Department of Electrical, Computer, & Biomedical Engineering
Faculty of Engineering & Architectural Science

Course Title:	Distributed Cloud Computing
Course Number:	COE 892
Semester/Year (e.g.F2016)	W2026

Instructor:	Muhammad Jaseemuddin

Assignment/Lab Number:	2
Assignment/Lab Title:	gRPC

Submission Date:	02/03/2026
Due Date:	06/03/2026

Student LAST Name	Student FIRST Name	Student Number	Section	Signature*
Meng	Alex	501095337	07	A.M

*By signing above you attest that you have contributed to this written lab report and confirm that all work you have contributed to this lab report is your own work. Any suspicion of copying or plagiarism in this work will result in an investigation of Academic Misconduct and may result in a "0" on the work, an "F" in the course, or possibly more severe penalties, as well as a Disciplinary Notice on your academic record under the Student Code of Academic Conduct, which can be found online at: http://www.ryerson.ca/senate/current/pol60.pdf

Introduction

In this lab, the mine-defusing robot scenario from Lab 1 is extended by implementing gRPC in the program. Rovers now communicate directly with a ground control server, which provides them with map data and commands. The remote procedure calls for this server are defined in a protobuf file.

Proto File

The full code for this file is located in the appendix. This file defines the interface for communication between the ground control server and the rovers. It specifies the RPC calls available to the rovers and their structures.

```
// Ground Control service for rovers
service ground_control {
    // Sends requested map to rover
    rpc getMap (mapRequest) returns (mapReply) {}
    // Sends command string with requested size to rover
    rpc getCommands (cmdReq) returns (cmdReply) {}
    // Sends serial number for the mine at the given position to rover
    rpc getSerial (serialRequest) returns (serialReply) {}
    // Receives signal for when command execution is done
    rpc reportStatus (status) returns (statusAck) {}
    // Gets a mine's PIN from the rover
    rpc sharePin (pinSend) returns (pinAck) {}
}
```

The ground control service defines 5 functions:

- getMap sends the rover the map matching the mapID in the request
- getCommands provides the rover with a string of commands matching the length specified in the request
- getSerial returns the serial number of the mine at the rover's current position
- reportStatus allows the rover to signal its status to the ground control server
- sharePin allows the rover to submit the valid pins it finds to the ground control server

Ground Control Server

The full code for this file is located in the appendix. This file implements the ground control server specified in the protobuf file. It's responsible for loading the map and mine data, and providing commands to the rovers. It also validates pins received from the rovers.

```python
def getMap(self, request, context):
        mapID = request.id
        filename = "map" + str(mapID) + ".txt"
        output = ""
        bounds = ""
        with open(filename, "r") as mapFile:
            bounds = mapFile.readline()
            output = mapFile.read()

        return groundControl_pb2.mapReply(map=output, bounds=bounds)
```

When a getMap request comes in, the server attempts to open the map file with the matching ID value. If it doesn't exist, it will return empty strings.

```python
def getCommands(self, request, context):
        return groundControl_pb2.cmdReply(commands=cmd(request.size))
def cmd(num: int):
    try:
        response = requests.get(URL + str(num))
        response.raise_for_status()
        return response.json()['commands']
    except requests.exceptions.RequestException as e:
        print("Error: " + str(e))
```

When a getCommands request comes in, the server calls the cmd function, which connects to the command-distributing server. It then sends the commands to the rover.

```python
def getSerial(self, request, context):
        filename = "mines" + str(request.id) + ".txt"
        contents = ""
        with open(filename, "r") as minesFile:
            contents = minesFile.read()
        contents = contents.split("\n")
        positions = contents[0].split(" ")
        count = 0
        for i in positions:
            if i == str(request.posX) + "," + str(request.posY):
```

```
                break
            count += 1
        return groundControl_pb2.serialReply(mineSerial=contents[count+1])
```

When a getSerial request comes in, the server opens the mines.txt file matching the map ID and provides the serial number of the mine at the rover's current location.

```
def reportStatus(self, request, context):
        if request.success == True:
            print("Rover " + str(request.id) + " finished defusing all mines")
        else:
            print("Rover " + str(request.id) + " exploded")
        return groundControl_pb2.statusAck(ack=True)
```

The reportStatus function just acknowledges status updates from rovers.

```
def sharePin(self, request, context):
        roverID = request.id
        pin = request.pinVal
        serial = request.serial
        print("Rover" + str(roverID) + " submitted PIN " + str(pin) + " for
serial " + str(serial))
        if valid_pin(serial, pin):
            print("Mine " + serial + " disarmed by rover " + roverID)
            return groundControl_pb2.pinAck(success=True)
        return groundControl_pb2.pinAck(success=False)

def valid_pin(serial, pin):
    key = serial + pin
    hashed = hashlib.sha256(key.encode()).hexdigest()

    return hashed.startswith("000000")
```

The sharePin function verifies that the submitted pin is valid using the valid_pin function, which checks if the hash output of the serial+pin string starts with 6 zeros.

Rover Client

The full code for this file is in the appendix. This file implements the rover client logic and its communications with the ground control server. It starts by initializing a connection and requesting map data from the server. Once it has the map data, it begins requesting commands from the server and executes them accordingly. When it receives a “dig” command, it checks if there is a mine there first, and then requests its serial number from the server if there is. If there is no mine there, then it ignores the command. Once the rover has received the serial number, it begins brute forcing the pin and sends the pin it finds to the server.

When the rover detects that it has been blown up or cleared all the mines from the field, it sends a status update to the server and ends.

Execution

Unsuccessful rover:

```
(.venv) PS C:\Users\alexm\OneDrive\Desktop\School Code\COE 892\Lab 2> & '
b 2/groundControl.py"
Rover 1 exploded
```

Successful rover:

```
(.venv) PS C:\Users\alexm\OneDrive\Desktop\School Code\
b 2/groundControl.py"
Rover1 submitted PIN XYZDDCBDFD5 for serial 25f78x
Rover1 submitted PIN XYZ371EAC28 for serial 0186x7
Rover 1 finished defusing all mines
```

Conclusion

In conclusion, this lab successfully extended the original mine-defusing robot system by introducing gRPC-based communication between the rovers and the ground control server. By defining the remote procedure calls in a protobuf file and implementing them on both the server and client sides, the system was transformed from a locally coordinated program into a distributed architecture with clearly defined service interfaces.

The ground control server centralized responsibility for map distribution, command retrieval, mine serial lookup, status monitoring, and PIN validation. Meanwhile, the rover clients focused on executing commands, detecting mines, brute-forcing valid PINs, and reporting outcomes. Overall, this lab reinforced key concepts in distributed systems, client-server

architecture, and remote procedure calls, while providing practical experience integrating networking with application logic.

Appendix A - Proto File

groundControl.proto:

```
syntax = "proto2";

// Ground Control service for rovers
service ground_control {
    // Sends requested map to rover
    rpc getMap (mapRequest) returns (mapReply) {}
    // Sends command string with requested size to rover
    rpc getCommands (cmdReq) returns (cmdReply) {}
    // Sends serial number for the mine at the given position to rover
    rpc getSerial (serialRequest) returns (serialReply) {}
    // Receives signal for when command execution is done
    rpc reportStatus (status) returns (statusAck) {}
    // Gets a mine's PIN from the rover
    rpc sharePin (pinSend) returns (pinAck) {}
}

// Map requests consists of the map ID being requested
message mapRequest {
    required string id = 1;
}

// Map reply sends map array as a string
// First row indicates number of rows and columns as space separated numbers
// Everything below is space separated array representing map.
message mapReply {
    required string map = 1;
    required string bounds = 2;
}

// Command requests give a size value to generate a command string of that
length
message cmdReq {
    required int32 size = 1;
}
```

```
// Returns a string of commands of the length specified in the request
message cmdReply {
    required string commands = 1;
}

// Serial number requests provide a mine's position in X and Y coordinates
message serialRequest {
    required int32 posX = 1;
    required int32 posY = 2;
    required int32 id = 3;
}

// Returns the serial number as a string
message serialReply {
    required string mineSerial = 1;
}

// Boolean indicating command execution success or failure
message status {
    required bool success = 1;
    required int32 id = 2;
}

// Acknowledgement of success or failure
message statusAck {
    required bool ack = 1;
}

// Pin value sent to server
message pinSend {
    required string pinVal = 1;
    required int32 id = 2;
    required string serial = 3;
}

// Acknowledgement that pin was recieved by server
message pinAck {
    required bool success = 1;
```

```
}
```

Appendix B - Ground Control Code

groundControl.py:

```
import grpc
import groundControl_pb2
import groundControl_pb2_grpc
import requests
import concurrent.futures as futures
import logging
import hashlib

URL = 'http://127.0.0.1:8000/lab1/rover/'
def cmd(num: int):
    try:
        response = requests.get(URL + str(num))
        response.raise_for_status()
        return response.json()['commands']
    except requests.exceptions.RequestException as e:
        print("Error: " + str(e))

def valid_pin(serial, pin):
    key = serial + pin
    hashed = hashlib.sha256(key.encode()).hexdigest()

    return hashed.startswith("000000")

class groundControl(groundControl_pb2_grpc.ground_controlServicer):
    def getMap(self, request, context):
        mapID = request.id
        filename = "map" + str(mapID) + ".txt"
        output = ""
        bounds = ""
        with open(filename, "r") as mapFile:
            bounds = mapFile.readline()
            output = mapFile.read()

        return groundControl_pb2.mapReply(map=output, bounds=bounds)
```

```python
    def getCommands(self, request, context):
        return groundControl_pb2.cmdReply(commands=cmd(request.size))

    def getSerial(self, request, context):
        filename = "mines" + str(request.id) + ".txt"
        contents = ""
        with open(filename, "r") as minesFile:
            contents = minesFile.read()
        contents = contents.split("\n")
        positions = contents[0].split(" ")
        count = 0
        for i in positions:
            if i == str(request.posX) + "," + str(request.posY):
                break
            count += 1
        return groundControl_pb2.serialReply(mineSerial=contents[count+1])

    def reportStatus(self, request, context):
        if request.success == True:
            print("Rover " + str(request.id) + " finished defusing all mines")
        else:
            print("Rover " + str(request.id) + " exploded")
        return groundControl_pb2.statusAck(ack=True)

    def sharePin(self, request, context):
        roverID = request.id
        pin = request.pinVal
        serial = request.serial
        print("Rover" + str(roverID) + " submitted PIN " + str(pin) + " for
serial " + str(serial))
        if valid_pin(serial, pin):
            print("Mine " + serial + " disarmed by rover " + roverID)
            return groundControl_pb2.pinAck(success=True)
        return groundControl_pb2.pinAck(success=False)

def serve():
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))
```

```
groundControl_pb2_grpc.add_ground_controlServicer_to_server(groundControl(),
server)
    server.add_insecure_port('[::]:50051')
    server.start()
    server.wait_for_termination()

if __name__ == '__main__':
    logging.basicConfig()
    serve()
```

Appendix C - Rover Client

roverClient.py

```
import grpc
import groundControl_pb2
import groundControl_pb2_grpc
import hashlib
import sys
import random

NORTH = 0
EAST = 1
SOUTH = 2
WEST = 3
initial_x = 0
initial_y = 0

initial_dir = SOUTH
addons = ["0", "1", "2", "3", "4", "5", "6", "7", "8", "9", "A", "B", "C", "D",
"E", "F"]

def run_hash(serial):
    while True:
        pin = "XYZ"
        for count in range(8):
            pin = pin + random.choice(addons)
        temp_key = pin + serial
        hashed = hashlib.sha256(temp_key.encode('utf-8')).hexdigest()
        zeroes = 0
        for ch in hashed:
            if ch == "0":
                zeroes += 1
            else:
                break
            if zeroes >= 6:
                return pin

def run(roverID: int, mapID: str):
```

```python
    channel = grpc.insecure_channel("localhost:50051")
    stub = groundControl_pb2_grpc.ground_controlStub(channel)

    # get map
    map_response = stub.getMap(groundControl_pb2.mapRequest(id=str(mapID)))
    y_bound, x_bound = map_response.bounds.split(" ")
    y_bound = int(y_bound)
    x_bound = int(x_bound)
    map_arr = []
    mines = 0
    for row in map_response.map.split("\n"):
        r = []
        for i in row.split(' '):
            r.append(int(i))
            if int(i) != 0:
                mines += 1
        map_arr.append(r)

    dir: int = initial_dir

    x = initial_x
    y = initial_y

    out_arr = [['0'] * x_bound for r in range(y_bound)]
    cmdIndex = 0
    commands = ["L", "M", "D", "R", "M", "M", "R", "M", "D"]
    while mines > 0:

        #cmd = stub.getCommands(groundControl_pb2.cmdReq(size=1)).commands
        cmd = commands[cmdIndex]
        cmdIndex += 1

        mine = False

        if cmd == 'R':
            if dir+1 == 4:
                dir = 0
            else:
```

```
                dir += 1
        elif cmd == 'L':
            if dir-1 == -1:
                dir = 3
            else:
                dir -= 1
        elif cmd == 'M':
            if map_arr[y][x] == 1:
                mine = True
            if dir == NORTH:
                if y-1 >= 0:
                    y -= 1
            if dir == EAST:
                if x+1 < x_bound:
                    x += 1
            if dir == SOUTH:
                if y+1 < y_bound:
                    y += 1
            if dir == WEST:
                if x-1 >= 0:
                    x -= 1
        elif cmd == 'D':
            print("Dig at " + str(x) + ", " + str(y))
            # get mine's serial number
            if map_arr[y][x] != 0:
                serial = stub.getSerial(groundControl_pb2.serialRequest(posX=x,
posY=y, id=mapID)).mineSerial
                print(serial)
                pin = run_hash(serial)
                print(pin)
                print(roverID)
                # mine successfully disarmed
                map_arr[y][x] = 0
                mines -= 1
                send_pin =
stub.sharePin(groundControl_pb2.pinSend(pinVal=str(pin), id=int(roverID),
serial=str(serial)))
```

```
                if send_pin.success:
                    print("Pin Acknowledge: " + send_pin.message)

        # check for mine, check for dig. If dig is false and mine is true,
break.

        out_arr[y][x] = '*'
        if mine is True:
            print("Boom")
            stub.reportStatus(groundControl_pb2.status(success=False,
id=int(roverID)))
            break

    with open("path_" + str(1) + ".txt", "w") as out:
        for r in out_arr:
            for pos in r:
                out.write(pos)
            out.write('\n')

    if mines == 0:
        stub.reportStatus(groundControl_pb2.status(success=True,
id=int(roverID)))


if __name__ == '__main__':
    if len(sys.argv) != 3:
        print("Proper usage: python roverClient.py [mapID] [roverID]")
    elif len(sys.argv) == 3:
        run(sys.argv[1], int(sys.argv[2]))
```